

PYTHON

PROYECTO FINAL

Viktor Piniehin



VIKTOR PINIEHIN

RESPONSABLE DEL DESARROLLO DE LA APLICACIÓN:

TECH-IT: Plataforma Integral de Comercio Electrónico
y Sistema Gestión de Inventario

Fecha: 10/06/2026



ÍNDICE DE CONTENIDOS

1. DESCRIPCIÓN GENERAL DEL PROYECTO	Pág. 3
• 1.1. Introducción (como apareció “Tech-IT”)	
• 1.2 Arquitectura del Sistema	
2. OBJETIVOS Y ALCANCE DEL PROYECTO	Pág. 4
• 2.1. Objetivos	
• 2.2. Alcance	
3. STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS	Pág. 5
• 3.1. Stack tecnológico	
• 3.2. Evaluación de Soluciones Alternativas	
4. MODELO DE DATOS.....	Pág. 6
• 4.1. Explicación de la Base de Datos	
• 4.2. Diagrama de Entidad-Relación de la Base de Datos	
5. EXPLICACIÓN DE LOS REQUISITOS DE LA APLICACIÓN	Pág. 7
• Funcionales	
• No funcionales	
6. MANUAL DE INSTALACIÓN Y CONFIGURACIÓN	Pág. 22
7. CONCLUSIONES	Pág. 23
• 7.1. Valoración General del Proyecto Finalizado	
• 7.2. Aprendizajes y experiencias adquiridas	
8. EVOLUTIVOS DEL PROYECTO	Pág. 26

DESCRIPCIÓN GENERAL DEL PROYECTO

“Desde la propuesta A - hasta un proyecto de temática libre”

El proyecto Tech-IT nació de forma espontánea mientras leía las diferentes propuestas para el proyecto final. Desde el primer momento, quise decantarme por la Propuesta A: “*Empresa de suministros tecnológicos*”, y empecé a imaginar qué aspecto y qué “personalidad” tendría la plataforma web de mi empresa.

Navegando a diario por internet, se observa que la gran mayoría de las tiendas online presentan una funcionalidad muy lineal, es decir, un conjunto estático de productos con sus fotos y descripciones. Al analizar este panorama, sentí la motivación de crear algo mucho más dinámico e interesante; un sitio web con un ambiente específico y una identidad propia, diseñado para que el cliente que visite la plataforma se sumerja en su atmósfera desde el primer segundo, por supuesto sin descuidar en ningún momento la robustez funcional.

Dado que la temática del proyecto gira en torno a los productos de tecnologías de la información (IT), decidí crear un sitio web animado con una marcada estética e influencia del estilo *cyberpunk*. Para lograr este impacto visual y conceptual, el ecosistema se introduce mediante un tráiler cinematográfico que establece el tono de la marca (más 2 tráilers para paginas de portátiles y smartphones). De este modo, Tech-IT no solo se presenta como una tienda online, sino como una experiencia digital envolvente para el usuario.

1.2. Arquitectura Comercial y Administrativa

El sistema se divide lógicamente en dos grandes áreas funcionales que interactúan de manera constante:

Frontend: Diseñada bajo la estética conceptual de corte tecnológico y futurista. Permite a los usuarios acceder a un catálogo clasificado dinámicamente por categorías (Smartphones y Portátiles) y segmentado por marcas comerciales mediante consultas directas a la base de datos. Incorpora un sistema de carrito de compra dinámico basado en la persistencia de datos por sesiones del navegador, lo que permite añadir, restar y eliminar unidades de productos en tiempo real, calculando los importes totales sin necesidad de un registro previo obligatorio.

Backend: Representa el núcleo operativo y de control del proyecto, protegido bajo un estricto protocolo de seguridad mediante decoradores de restricción de acceso (@login_required). A través de una interfaz de inicio de sesión personalizada, el personal autorizado puede acceder a un cuadro de mando o *dashboard* de gestión empresarial. Este panel administrativo otorga el control total sobre el inventario y permite la exportación instantánea del stock de la empresa en documentos PDF.

La integración de ambas áreas se realiza de forma transparente a través del mapeador de datos relacional (ORM) de Django. Cualquier modificación efectuada en el inventario o en la base de datos por parte de los administradores se refleja en la interfaz comercial pública que visualiza el cliente final.



OBJETIVOS Y ALCANCE DEL PROYECTO

Objetivos :

- Desarrollar una aplicación web funcional y escalable basada en Django.
- Diseñar una experiencia de usuario (UX/UI) inmersiva y diferenciadora mediante una estética *cyberpunk*, demostrando que las aplicaciones de gestión comercial pueden convivir con identidades visuales creativas y dinámicas sin perder usabilidad.
- Implementar un sistema de gestión interna seguro que centralice las tareas administrativas de control de inventario y optimice el flujo de trabajo del personal.
- Garantizar la persistencia de datos en el cliente mediante la gestión de sesiones en Django para el carrito de compras, logrando que sume, reste y elimine productos en tiempo real con un cálculo preciso de costes.
- Garantizar la seguridad del módulo administrativo restringiendo el acceso mediante decoradores de autenticación y construyendo un formulario de login personalizado con validaciones visuales de errores.
- Automatizar la generación de reportes corporativos mediante la integración de la librería ReportLab, permitiendo compilar el stock actual de la base de datos en un documento PDF formateado con un solo clic.

Alcance:

El alcance de Tech-IT incluye el desarrollo de un catálogo dinámico filtrado por categorías y marcas, junto con un carrito de compra basado en sesiones del servidor. Asimismo, contempla un módulo administrativo protegido por contraseña que centraliza el control del inventario y permite la exportación automática de informes de stock en formato PDF. Por el contrario, se excluyen de esta fase el desarrollo de una pasarela de pago real con entidades bancarias y el registro público para clientes compradores. De este modo, el sistema se acota a una solución funcional de comercio electrónico y gestión interna empresarial en un entorno local seguro.



STACK TECNOLÓGICO Y ALTERNATIVAS EVALUADAS

Stack Tecnológico

- Lenguaje de Programación - Python 3.13.7
- Framework Web - Django 6.0.2
- Base de Datos - SQLite 3
- Librerías de Terceros: ReportLab (generación de Informes), Pillow 12.2.0 (gestión de Imágenes), sqlparse 0.5.5 (procesamiento de Consultas)
- Entorno de Desarrollo - PyCharm

Evaluación de Soluciones Alternativas

Alternativa al Framework: Flask v/s Django

Ventajas de Flask: Ofrece una libertad total para estructurar el proyecto y carece de la rigidez de carpetas iniciales de Django.

Desventajas de Flask: No incluye un sistema de administración nativo, ni ORM, ni herramientas de autenticación de fábrica. Para construir una plataforma como Tech-IT, habría sido necesario instalar, configurar y testear múltiples librerías de terceros independientes.

Decisión: Se descartó Flask y se eligió Django porque proporciona todas estas herramientas integradas y testeadas de forma nativa, garantizando la seguridad del panel administrativo y acelerando el tiempo de desarrollo. (más detallado en conclusiones)

Alternativa a la Base de Datos: PostgreSQL v/s SQLite

Ventajas de PostgreSQL: Alta concurrencia, excelente gestión de volúmenes masivos de datos y herramientas avanzadas para producción en la nube.

Desventajas de PostgreSQL: Requiere la instalación, configuración y mantenimiento de un servidor de base de datos local independiente, lo que añade una capa de complejidad innecesaria en la fase de desarrollo académico. Además, dificulta la portabilidad del proyecto.

Decisión: Se eligió SQLite porque, para el alcance del proyecto y la entrega en un único archivo comprimido, su portabilidad es insuperable. Permite que el proyecto funcione de inmediato en cualquier máquina con solo arrancar el servidor de Django, manteniendo una estructura limpia y funcional.

MODELO DE DATOS

Explicación de la Base de Datos

Para el proyecto Tech-IT, se ha seleccionado SQLite 3 como el motor de base de datos relacional principal. A diferencia de los sistemas de gestión de bases de datos tradicionales (MySQL , PostgreSQL) que operan bajo una arquitectura de servidor independiente, SQLite funciona como una base de datos incrustada - directamente dentro de la propia aplicación (más simple).

Base de datos de Tech-IT cuenta con 15 tablas y 17 índices, se almacenan físicamente dentro de un único archivo plano llamado db.sqlite3, ubicado en la raíz del proyecto.

Las tablas están relacionadas por Foreign Key (proveedor_id “relaciona” las tablas Producto y Proveedor)

Cuando el servidor de Django necesita leer o escribir datos - realiza llamadas directas al archivo lo que elimina latencia de comunicación por red corporativa.

Diagrama de Entidad-Relación de la Base de Datos



REQUISITOS DE LA APLICACIÓN

Requisitos Funcionales:

1. Introducción (Tráiler Cinematográfico) – son 3 para la pagina principal (intro), la pagina de smartphones (árbol) y portátiles (almacén)

Descripción: la bienvenida al usuario mediante tráiler cinematográfico que establezca la atmósfera visual y la identidad conceptual de la marca *cyberpunk* antes de dar acceso al catálogo comercial.

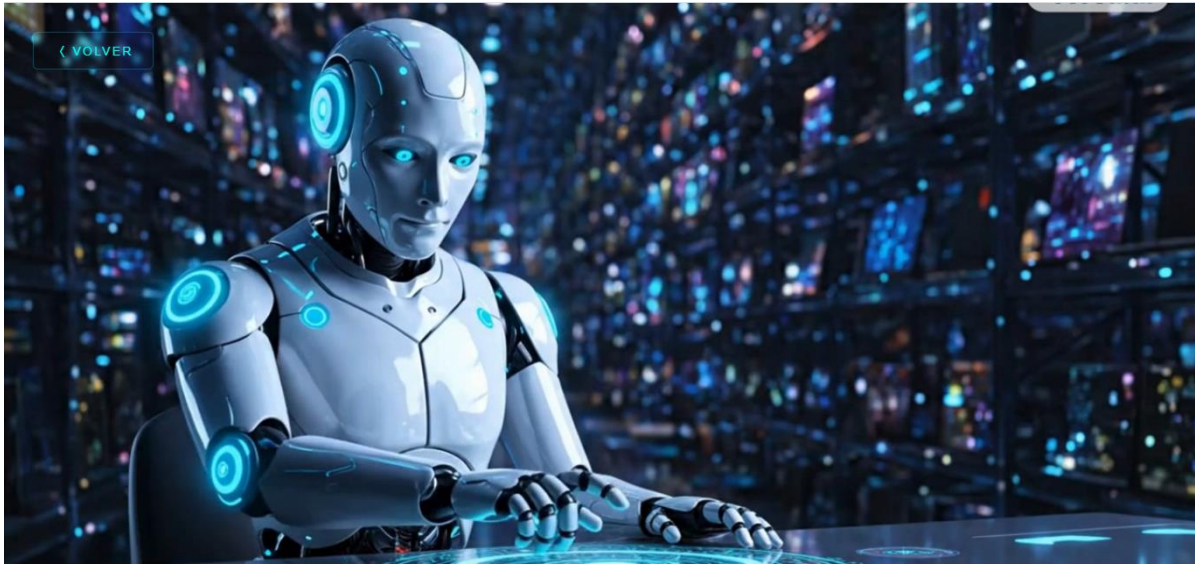
Intro:



Árbol:



Almacén:



2. Catálogo Dinámico Segmentado por Categorías y Marcas (menú)

La interfaz permite libre navegación y el filtrado dinámico de los artículos según su categoría principal (Smartphones o Portátiles) y la marca seleccionada en el menú.

Menú principal:



Menú Smartphones



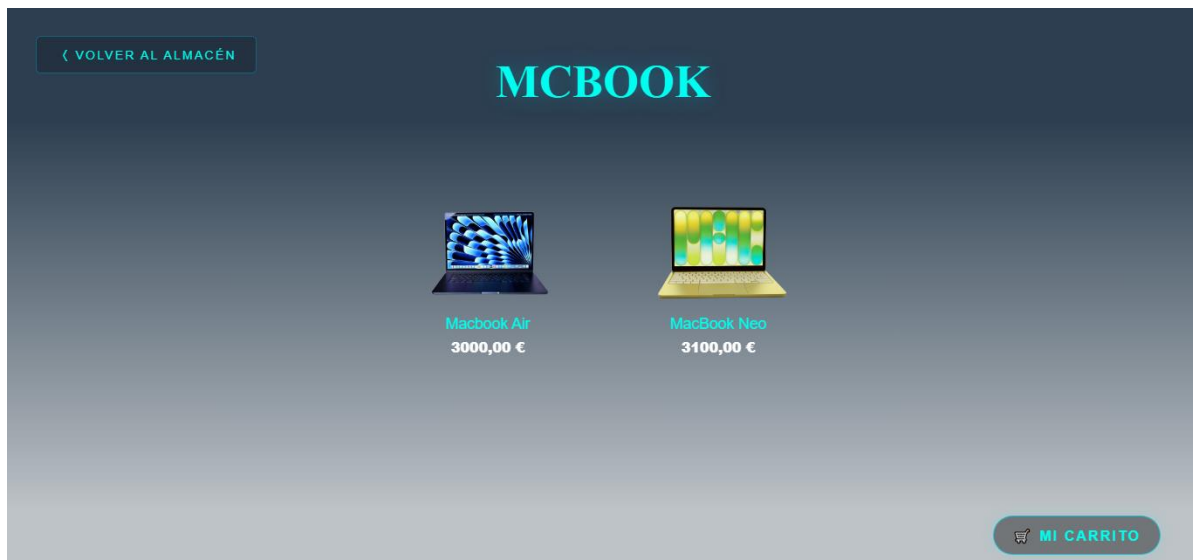
Menú Smartphones 2



Menú Portátiles



Menú Portátiles2



4. Carrito de Compra

El sistema permite a los usuarios añadir productos al carrito de compra, incrementar o decrementar las unidades de un artículo y eliminarlos por completo. La pantalla del carrito calcula de forma automática los subtotales por producto y el importe total facturable de la orden.

Carrito con productos añadidos:



The screenshot displays a shopping cart interface titled "SISTEMA DE PEDIDOS | MI CARRITO". It features a table with the following columns: PRODUCTO, NOMBRE, PRECIO, CANTIDAD, TOTAL, and ACCIONES. Two items are listed: a Zenbook laptop and an iPhone Air. Below the table, there is a "VACIAR SISTEMA" button and a "TOTAL GENERAL: 3600,0 €" label. A "PROCESAR PEDIDO →" button is located at the bottom right. The URL "127.0.0.1:8000/checkout/" is visible in the bottom left corner.

PRODUCTO	NOMBRE	PRECIO	CANTIDAD	TOTAL	ACCIONES
	Zenbook	2400.00 €	1	2400.00 €	  
	iPhone Air	1200.00 €	1	1200.00 €	  

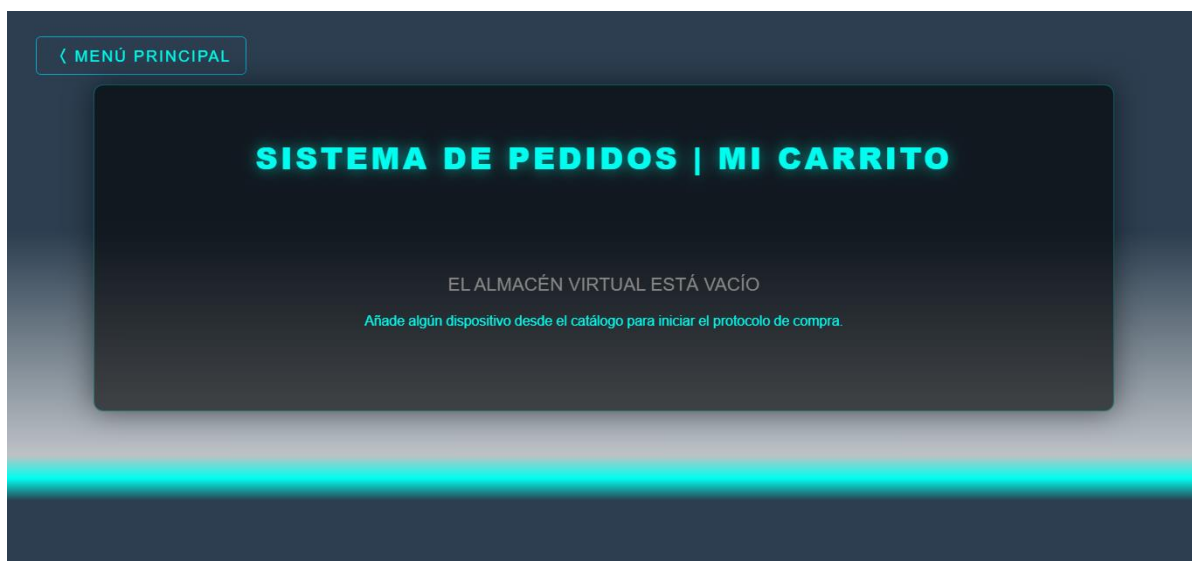
VACIAR SISTEMA

TOTAL GENERAL: 3600,0 €

PROCESAR PEDIDO →

127.0.0.1:8000/checkout/

Carrito vacío:



The screenshot displays the same shopping cart interface, but it is empty. A message in the center states "EL ALMACÉN VIRTUAL ESTÁ VACÍO" and "Añade algún dispositivo desde el catálogo para iniciar el protocolo de compra." A "← MENÚ PRINCIPAL" button is visible in the top left corner.

← MENÚ PRINCIPAL

SISTEMA DE PEDIDOS | MI CARRITO

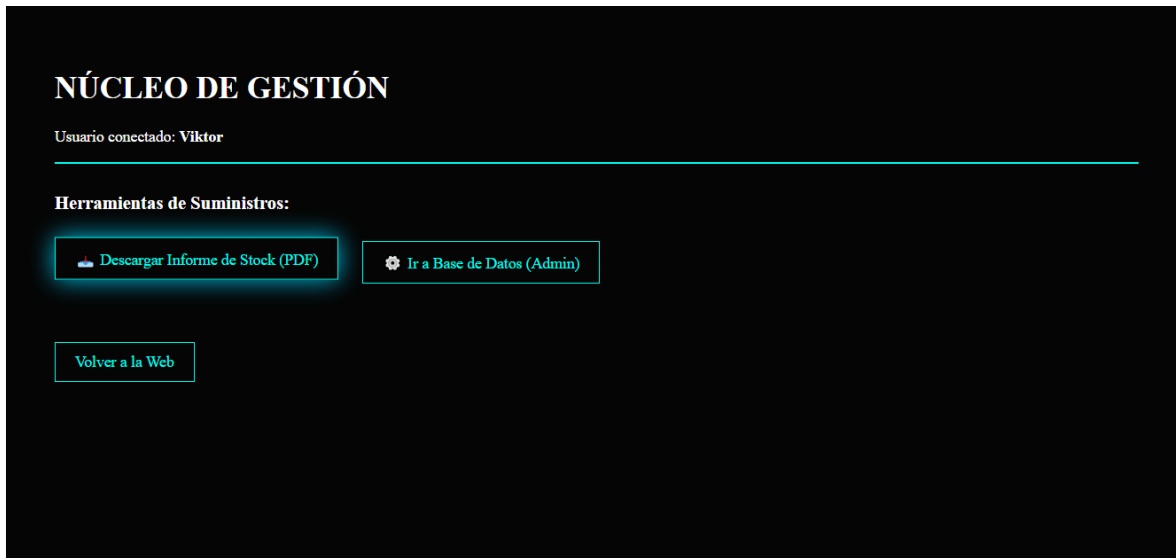
EL ALMACÉN VIRTUAL ESTÁ VACÍO

Añade algún dispositivo desde el catálogo para iniciar el protocolo de compra.

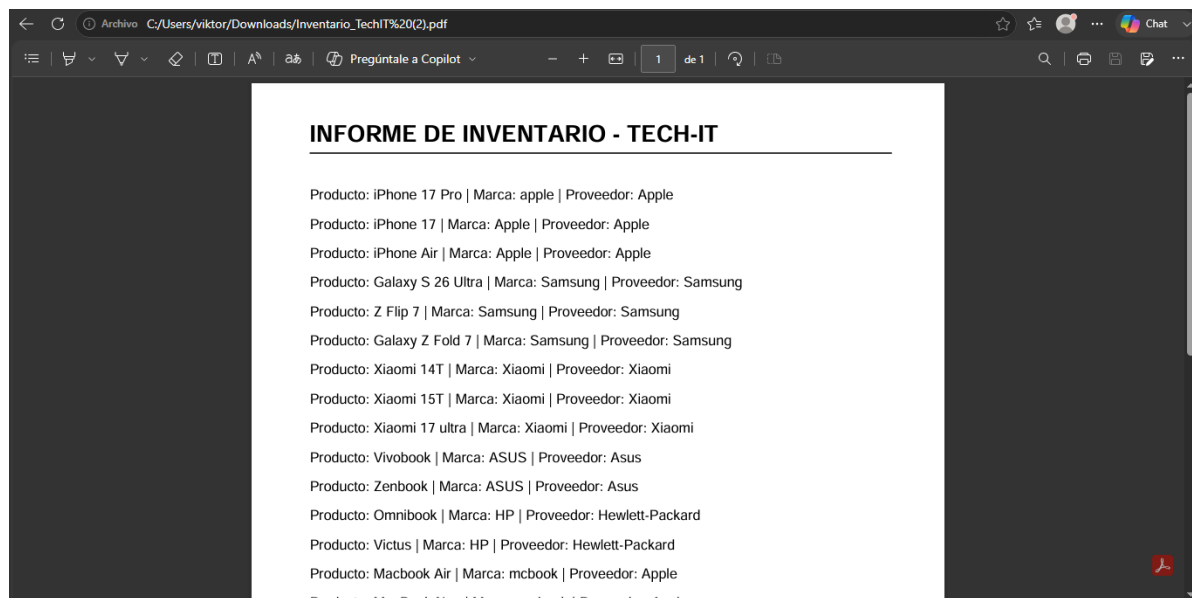
5. Dashboard de Control de Inventario Empresarial

Para auditar el inventario global de la empresa, supervisar las existencias y controlar los datos corporativos relacionales.

Dashboard:



PDF descargado:



6. Búsqueda de Productos

Búsqueda en la cabecera de la interfaz que permita a los usuarios localizar dispositivos introduciendo términos clave (como parte del nombre o la marca). El sistema debe procesar la cadena de texto y devolver únicamente los productos coincidentes.

Busqueda por “17”



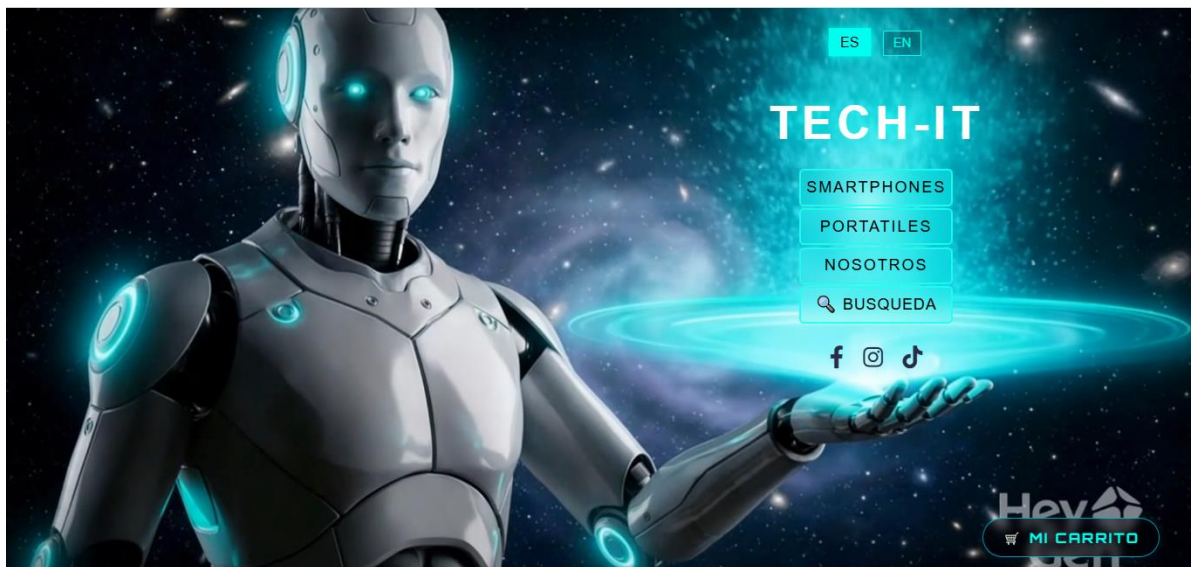
Resultados:



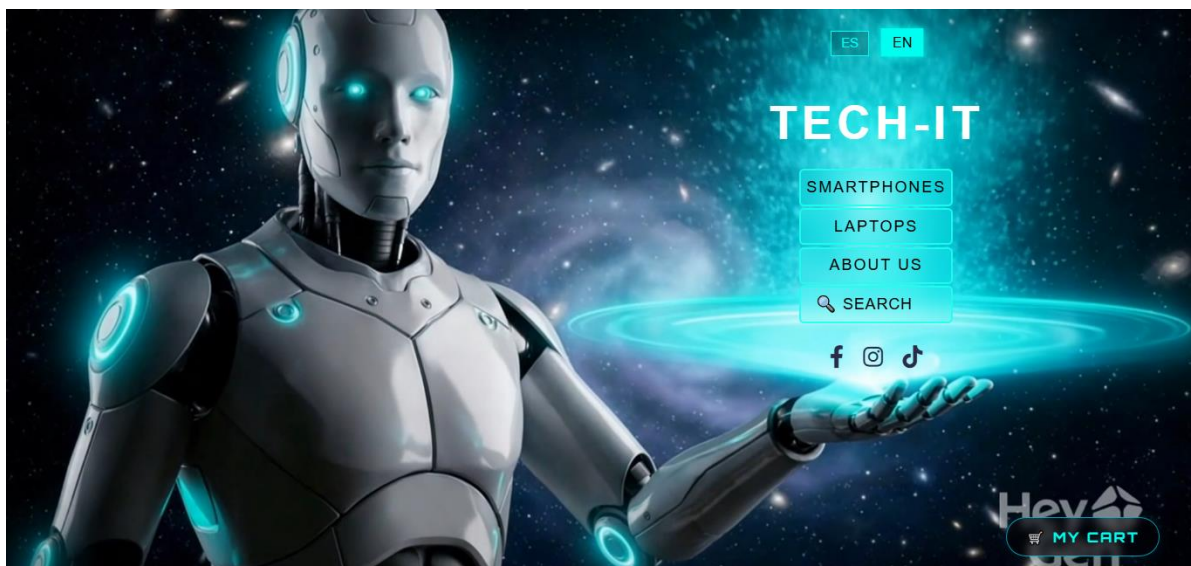
7. Cambio del idioma

Se puede cambiar el idioma (es - en), se puede añadir cualquier idioma

Es:



En:



8. Elances a las redes sociales

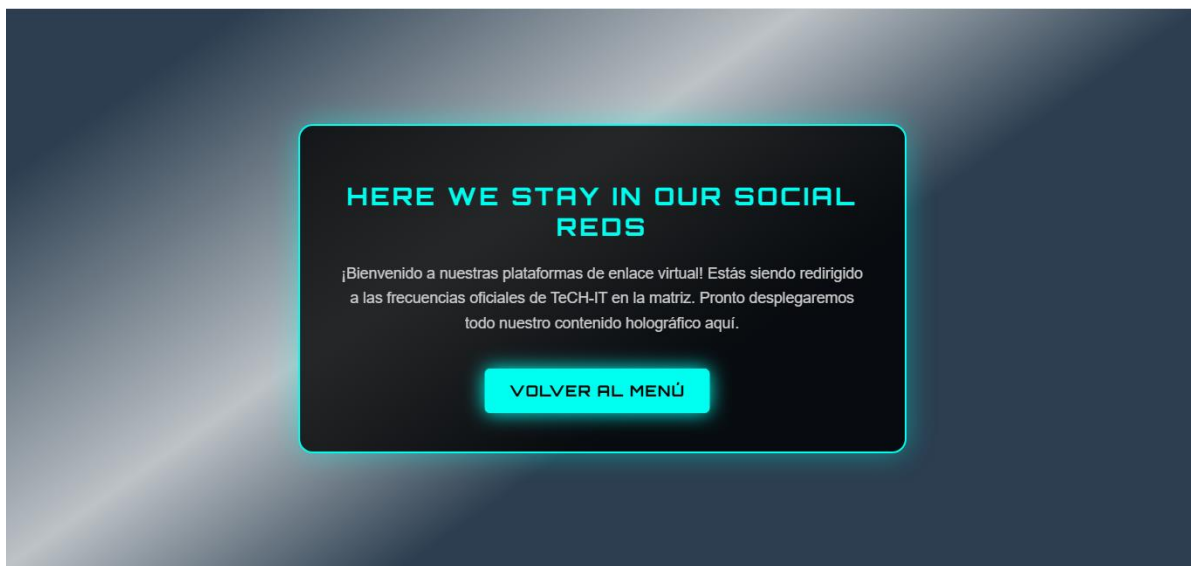
Los accesos directos a los perfiles corporativos oficiales de las redes sociales de la empresa, permitiendo la fidelización del cliente fuera del ecosistema web.

(solo 3 redes pero se puede añadir más – es solamente para marcar la posibilidad)

Menú redes:



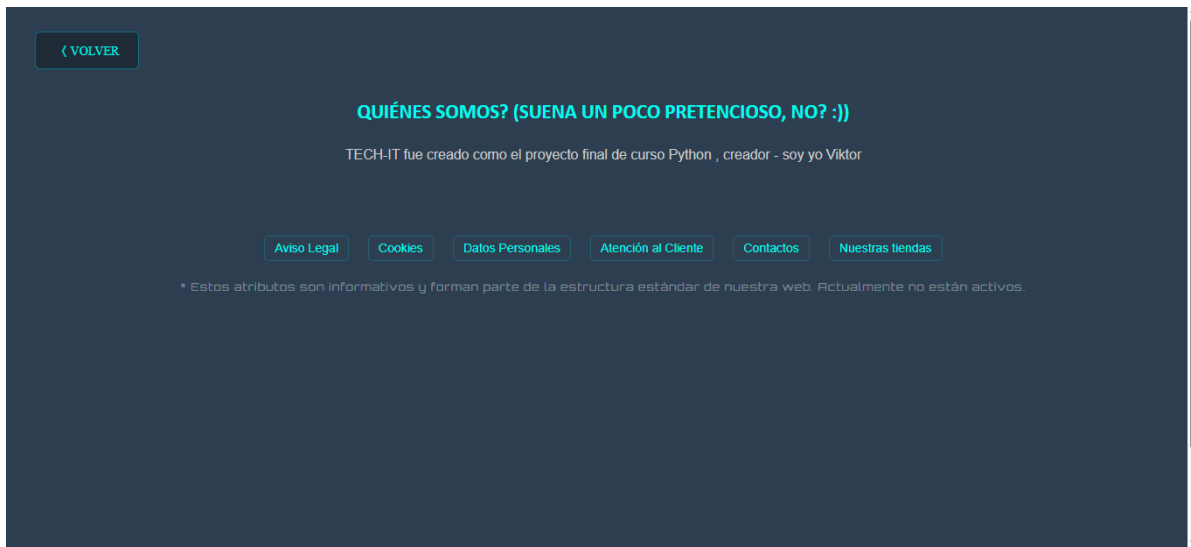
Página:



9. NOSOTROS

Una sección corporativa ("Nosotros") donde se explique la historia, misión y valores de la empresa bajo el contexto visual *cyberpunk*. Estos textos deben cargarse dinámicamente para que puedan ser modificados sin alterar el código fuente.

Nosotros:



Como se pone en captura de pantalla todos los botones no están activos

10. Procesamiento y Confirmación de Pedidos (Simulación de Checkout)

Permitir al usuario formalizar los artículos acumulados en su carrito de compras a través de una pasarela de confirmación integrada (checkout.html). Al finalizar el proceso de pedido, el sistema debe vaciar automáticamente el carrito del usuario y mostrar una pantalla de éxito o confirmación con la estética del ecosistema.

Pedido1:



The screenshot displays a checkout interface with a dark theme. On the left, there are input fields for 'NOMBRE COMPLETO' (Dan Brain), 'DIRECCIÓN DE ENTREGA' (Das Fontiñas, 5), 'CIUDAD' (Lugo), and 'CÓDIGO POSTAL' (27002). Below these is a section titled '2. MÉTODO DE PAGO (SIMULACIÓN SECURE-TEC)' containing fields for 'NÚMERO DE TARJETA' (7845 2145 3698 7489), 'FECHA CADUCIDAD' (05/29), and 'CÓDIGO CVV' (541). A large red button labeled 'EJECUTAR PAGO SEGURO' is at the bottom. On the right, a summary box lists items: 'Vivobook (x1)' for 2500.00 €, 'iPhone 17 Pro (x1)' for 2000.00 €, and 'Xiaomi 17 ultra (x1)' for 1800.00 €, with a total of 6300,0 €.

Item	Price
Vivobook (x1)	2500.00 €
iPhone 17 Pro (x1)	2000.00 €
Xiaomi 17 ultra (x1)	1800.00 €
TOTAL A PAGAR:	6300,0 €

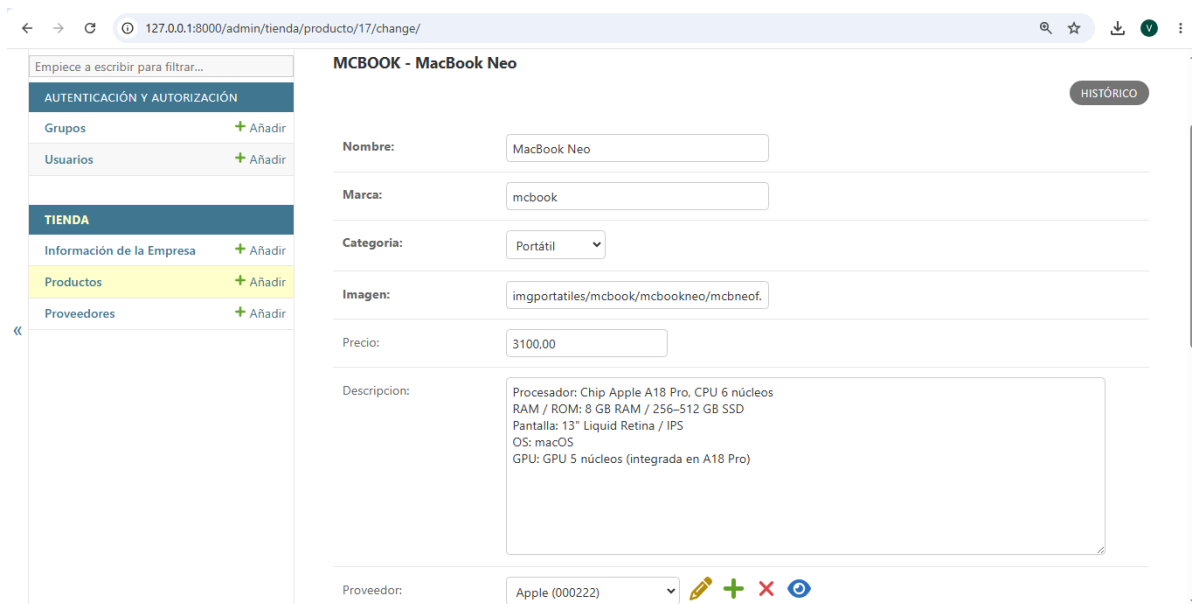
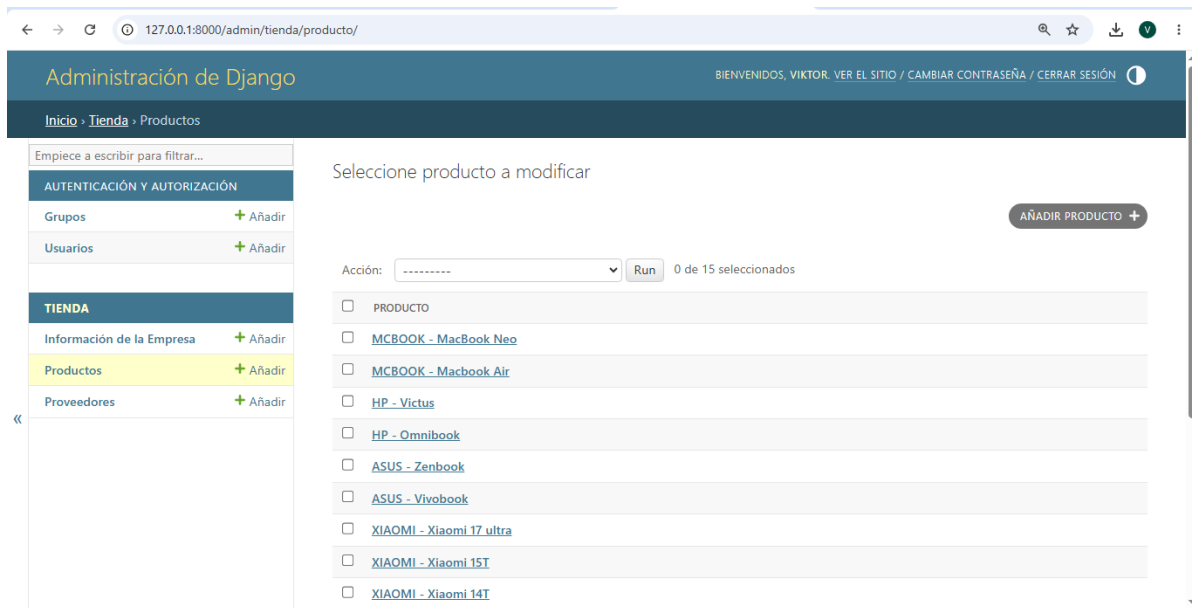
Pedido2:



The screenshot shows a confirmation screen with a dark background. At the top is a large green checkmark. Below it, the text 'PROTOCOLO DE PAGO COMPLETADO' is displayed in bold. A message follows: '¡Muchas gracias por tu compra, Dan Brain! El sistema ha verificado la transacción virtual de manera satisfactoria. Tus dispositivos de última generación ya están listos en el hangar de envío.' Below this is a box containing the 'CÓDIGO SECURE-ID: #TECH-1209600'. At the bottom is a red button labeled 'VOLVER AL MENÚ PRINCIPAL'.

11. Panel de Administración Centralizado (Django Admin)

Exclusivamente para los usuarios con rango de staff o superusuarios. Desde esta interfaz, el administrador puede realizar operaciones CRUD completas de forma directa sobre las tablas del catálogo, los proveedores y la información institucional de la empresa.



← → ↻ 127.0.0.1:8000/admin/tienda/proveedor/ 🔍 ☆ ⬇️ 🌱 ⋮

Administración de Django BIENVENIDOS, VIKTOR. VER EL SITIO / CAMBIAR CONTRASEÑA / CERRAR SESIÓN

Inicio > Tienda > Proveedores

Empiece a escribir para filtrar...

AUTENTICACIÓN Y AUTORIZACIÓN

Grupos + Añadir

Usuarios + Añadir

TIENDA

Información de la Empresa + Añadir

Productos + Añadir

Proveedores + Añadir

Seleccione Proveedor a modificar

AÑADIR PROVEEDOR +

🔍 Buscar

Acción: Run 0 de 5 seleccionados

☐	NOMBRE DE LA EMPRESA	NIF / CIF	PERSONA DE CONTACTO	TELÉFONO
☐	Asus	000555	Pedro	00000000005
☐	Hewlett-Packard	000444	Mr Been	00000000004
☐	Xiaomi	000333	Lu Kan	00000000003
☐	Apple	000222	Djim Carry	00000000002
☐	Samsung	000111	Kun Lao	00000000001

5 Proveedores

← → ↻ 127.0.0.1:8000/admin/tienda/producto/17/change/ 🔍 ☆ ⬇️ 🌱 ⋮

Empiece a escribir para filtrar...

AUTENTICACIÓN Y AUTORIZACIÓN

Grupos + Añadir

Usuarios + Añadir

TIENDA

Información de la Empresa + Añadir

Productos + Añadir

Proveedores + Añadir

IMAGEN PRODUCTOS

IMAGEN ¿ELIMINAR?

Imagen de MacBook Neo

imgportatiles/mcbook/mcbookneo/mcbneof ☐

Imagen de MacBook Neo

imgportatiles/mcbook/mcbookneo/mcbneok ☐

Imagen de MacBook Neo

imgportatiles/mcbook/mcbookneo/mcbneop ☐

✕

✕

✕

+ Agregar Imagen producto adicional.

GUARDAR Guardar y añadir otro Guardar y continuar editando Eliminar



Requisitos No Funcionales:

- 1. Seguridad y Protección de Datos:** La aplicación protege la información confidencial de manera automática. Las contraseñas de los usuarios no se guardan en texto plano, sino encriptadas en la base de datos mediante el sistema de seguridad nativo de Django
- 2. Portabilidad y Facilidad de Despliegue:** El proyecto es completamente independiente y "listo para usar". Al utilizar SQLite 3 como motor integrado, toda la base de datos viaja guardada dentro del propio archivo db.sqlite3. Esto elimina la necesidad de instalar o configurar servidores externos.
- 3. Gestión Estandarizada de Librerías:** El entorno de desarrollo está totalmente controlado y documentado en el archivo requirements.txt



MANUAL DE INSTALACIÓN

Este apartado describe el procedimiento técnico necesario para desplegar de forma local el entorno de desarrollo y ejecución de la plataforma Tech-IT en sistemas operativos Windows.

Paso 1: Instalar Python

1. Entra en python.org/downloads y descárgalo.
2. MUY IMPORTANTE: Al abrir el instalador, abajo del todo marca la casilla que dice Add python.exe to PATH

Paso 2: Descomprimir el proyecto

1. Descomprime el archivo TechIT_Final.zip en una carpeta que tengas a mano (por ejemplo, en el Escritorio).

Paso 3: Abrir la consola de comandos (Terminal)

1. Entra en la carpeta que acabas de descomprimir (tiene que ver el archivo manage).
2. Haz clic en la barra de direcciones de arriba (donde pone la ruta de la carpeta), borra todo, escribe cmd y pulsa Enter.
3. Se abrirá una ventana negra (la consola).

Paso 4: Crear el entorno virtual e instalar las librerías

Copia y pega estos comandos en la ventana negra uno a uno (pulsando Enter después de cada uno):

1. Crear la caja del proyecto:

```
python -m venv venv
```

(Espera unos segundos a que vuelva a aparecer la línea para escribir).

2. Activar la caja:

```
call venv\Scripts\activate
```

(Verás que ahora pone (venv) al principio de la línea).

3. Instalar el software necesario de forma automática:

```
pip install -r requirements.txt
```

(Verás que empiezan a descargarse cosas, espera a que termine).

Paso 5: Arrancar la web

En el Terminal :

```
python manage.py runserver
```

Paso 6: Ver el resultado

Aparece este elance:

Starting development server at <http://127.0.0.1:8000/> - pulsalo y bienvenido al Tech-IT.



CONCLUSIONES

Valoración General del Proyecto

Para entender el resultado de este proyecto, tengo que confesar por qué elegí Django. En el curso de Tokio School ya habíamos estudiado y aprendido cómo funcionaban herramientas como Tkinter (aplicaciones de escritorio) o Flask (framework de desarrollo web más simple). Habría sido más cómodo quedarme ahí, en mi “zona de confort”. Sin embargo, de Django no sabía absolutamente nada. Tenía curiosidad, quería aprenderlo de verdad y tengo claro que la mejor ruta para dominar una tecnología es tirarse a la piscina y practicar con un proyecto real. Por eso lo elegí.

Al avanzar en el desarrollo, me di cuenta de la enorme diferencia con Flask. Mientras que Flask es un micro-framework que viene prácticamente vacío y te obliga a buscar e instalar librerías externas para todo, Django es una herramienta mucho más potente y profesional. Trae incorporadas de serie funciones críticas que en Flask tendrías que programar desde cero: un panel de administración nativo ya listo para usar, un sistema de autenticación y seguridad perimetral de usuarios superrobusto, y un ORM integrado que gestiona de forma automática bases de datos complejas. Elegir Django significó subir un escalón hacia el estándar que se utiliza hoy en día en la industria del software profesional."

El resultado de ese desafío es Tech-IT, una plataforma que no se parece en nada a las tiendas online "estándar" que vemos todos los días en internet. En lugar de hacer una web plana, quise crear algo único: una experiencia inmersiva con una estética *cyberpunk* y su propia personalidad.

Lo que diferencia a esta web de las demás es su estilo animado y dinámico: desde el primer segundo, el usuario es recibido por un tráiler cinematográfico que introduce la marca, y toda la interfaz responde con colores vivos, menús fluidos y un diseño visualmente potente (al menos a mi me parece así). No es solo una base de datos con productos; es un entorno vivo donde el catálogo, el carrito de compras y el panel de administración (y el resto) se integran dentro de una ambientación futurista única.



Aprendizajes y experiencias adquiridas

Desarrollar Tech-IT no ha sido solo diseñar una interfaz; ha sido coordinar diferentes lenguajes de programación, patrones de diseño de software y herramientas de backend:

- **Programación Orientada a Objetos y Arquitectura de Datos (Python y Django):** He diseñado e implementado una estructura relacional sólida en la base de datos SQLite 3 mediante `models.py`, aplicando conceptos de herencia y abstracción:
 - **Relaciones de Datos Seguras:** Vinculé la tabla `Producto` con la de `Proveedor` mediante `ForeignKey (proveedor_id)`.
 - **Panel de Administración:** Registré los modelos en `admin.py` optimizando la experiencia de gestión. Mediante la clase `ImagenProductoInline (admin.TabularInline)`, permití la carga masiva y simultánea de múltiples imágenes (`extra=3`) para alimentar la galería dinámica de cada dispositivo.
 - **Interfaz Mixta:** Para no limitar la estética cyberpunk, aislé las páginas principales (`index.html`, `portatiles.html`, `smartphones.html`) para controlar de forma autónoma los estilos CSS a pantalla completa y las cargas de vídeo, mientras que para el catálogo y la gestión apliqué modularidad reutilizable con la etiqueta `{% include %}` (como en `selector_idiomas.html`).
- **Lógica del Servidor, Estado y Seguridad:** A través de `views.py` y `carrito.py`, programé la lógica de control del negocio:
 - **Control de Sesiones:** Desarrollé una clase `Carrito` nativa que encapsula la manipulación del diccionario de la sesión. Utilicé la propiedad `self.session.modified = True` para forzar la persistencia en el servidor en cada inserción, resta o limpieza de productos.
 - **Seguridad del Sistema:** Protegí el panel de administración (`dashboard_gestion`) mediante el decorador `@login_required` sincronizado con el parámetro `LOGIN_URL` en `settings.py`, impidiendo accesos no autorizados mediante inyección de URLs directas.
 - **Navegación:** Diseñé la vista `volver_a_origen` utilizando variables de sesión (`request.session['ultima_url']`) para almacenar dinámicamente las rutas absolutas analizadas (`build_absolute_uri()`), evitando el error de "doble click" al retroceder.
- **Internacionalización Avanzada (i18n):** Configuré el middleware global `LocaleMiddleware` en `settings.py` para habilitar el motor multilingüe. Implementé el enrutamiento bilingüe mediante `set_language` de Django, gestionando formularios con tokens `{% csrf_token %}` y capturando el parámetro oculto `next` para leer la ruta exacta (`request.get_full_path`). Esto permite cambiar instantáneamente entre español e inglés (hay francés pero no activada).
- **JavaScript y Animaciones de Rendimiento (Interactividad Eficiente):** Delegué en JavaScript el comportamiento en el cliente de manera optimizada:



- Combiné el bucle `{% for %}` de Django con variables CSS inyectadas en tiempo de renderizado (`{{ forloop.counter0|add:0.5 }}`s) para crear un retraso escalonado (`animation-delay`) en la carga de productos, logrando fluidez visual sin saturar el hilo principal del navegador.
- *Flujo Cinematográfico*: Utilicé el almacenamiento de sesión (`sessionStorage`) mediante claves independientes (`introVisto` y `almacenVisto`, `ArbolVisto`) para evitar la reproducción reiterada de los vídeos de fondo en una misma sesión.
- *Persistencia en Pasarela*: En `checkout.html` implementé la persistencia local (`localStorage`) para retener datos del usuario mientras se simulaba una llamada asíncrona de pago con un spinner interactivo.
- **Librerías del Ecosistema y Optimización (`requirements.txt`)**: Cada herramienta externa cumple un rol en el proyecto:
 - **ReportLab** (con optimización SQL): Utilizada para maquetar el inventario en formato PDF.
 - **Pillow**: Encargada del procesamiento y validación de las dimensiones de las imágenes del catálogo en el servidor.
 - **sqlparse**: Traduce de forma limpia el ORM de Django a sentencias SQL estructuradas para SQLite.
 - **asgiref**: Motor asíncrono subyacente que gestiona la concurrencia del servidor web.
 - **tzdata** y **charset-normalizer**: Garantizan la normalización UTF-8 para evitar la corrupción de caracteres especiales (como acentos o la letra "ñ").

Bueno, sea injusto dejar sin palabra la Inteligencia Artificial, hoy se usa casi en todo... Lejos de usar la IA para que "hiciera el trabajo por mí", la integré en mi flujo de trabajo en PyCharm como un consultor de arquitectura. Juntos fuimos estructurando el backend/frontend paso a paso, debatiendo la lógica de las vistas dinámicas, diseñando el sistema de sesiones del carrito para que fuera seguro y depurando los errores de rutas en las URLs o conflictos con los archivos estáticos en las hojas de estilo CSS. Cada paso (bueno, casi cada) pedí que me explica todo: por qué así? Por qué hacemos de esta manera y no de otra? AI – no es una cosa mágica que te permite resolver todos tus problemas y sin conocimientos (refiero de programación) te atrapas a un bucle indefinido resolviendo “conflictos” entre los archivos, a menudo me tocaba borrar los archivos y empezar de nuevo todo (`smartphones.html`, `portátiles.html` – muchas, muchas veces tenía que reescribirlos sin alguna consulta de AI). No hace falta olvidar que AI – es una herramienta muy potente y inevitable, pero es un ayudante.



EVOLUTIVOS DEL PROYECTO

Para transformar Tech-IT de un prototipo funcional a una plataforma de producción comercial, se puede cambiar/añadir lo siguiente :

1. Migración a Base de Datos de Producción (por ejemplo - PostgreSQL): Sustituir SQLite 3 por PostgreSQL que permite soportar un alto volumen de transacciones concurrentes y consultas SQL complejas de forma eficiente.
2. Pasarela de Pago Real e Integración de API (como PayPal): Reemplazar la simulación holográfica actual de checkout.html por la integración del SDK de PayPal, conectando la lógica del carrito con un sistema de transacciones financieras reales y seguras bajo cumplimiento PCI-DSS (*Payment Card Industry Data Security Standard* o Estándar de Seguridad de Datos para la Industria de Tarjetas de Pago).
3. Autenticación de Clientes Avanzada y Perfiles: Expandir el sistema de usuarios para permitir el registro e inicio de sesión de clientes. Esto incluirá historial de pedidos, seguimiento de envíos y pasarela de recuperación de contraseñas.
4. Sistema de Despliegue en la Nube (Docker & AWS): Contenerizar la aplicación completa utilizando Docker para asegurar la portabilidad del entorno de PyCharm y desplegar el servidor en servicios en la nube como Amazon Web Services, automatizando copias de seguridad y almacenamiento de archivos estáticos en Amazon S3.
5. Localización Geoespacial e Integración de API (Google Maps): Conectar la aplicación con la API de Google Maps mediante JavaScript y credenciales seguras de Google Cloud. El objetivo es desplegar un mapa interactivo personalizado con la estética visual de la web para geolocalizar los almacenes y laboratorios físicos de reparación de la empresa, calculando rutas en tiempo real para los clientes que prefieran la recogida en mano.
6. Soporte Multilingüe Escalable (Nuevos Idiomas): Ampliar el catálogo de traducciones en la carpeta locale generando los archivos de traducción .po y .mo para francés (ya preconfigurado en settings.py), alemán y japonés. Esto consolidará a Tech-IT como una plataforma de comercio electrónico global y completamente adaptada a mercados internacionales de alta tecnología.
7. Expansión de Canales y Estrategia Social Media (Redes Sociales): Desplegar la presencia de la marca en plataformas clave mediante la integración de APIs y SDKs oficiales.